

دليل البرمجة والتشغيل الشامل لأداة فحص الشبكات والثغرات

تطوير وتأمين الشبكات المحلية باستخدام لغة Python ومكتبة Scapy

⚠ إخلاء مسؤولية قانوني وأمني

إن هذا المستند مخصص بالكامل لأغراض التعليم، التدريب الأكاديمي، واختبار الاختراق المصرح به أخلاقياً (Ethical Penetration Testing). يمنع منعاً باتاً استخدام هذه الأكواد أو التقنيات المذكورة لفحص شبكات أو أجهزة. إساءة استخدام هذه الأدوات قد يعرضك للمساءلة القانونية بموجب قوانين مكافحة الجرائم المعلوماتية.

مقدمة عامة

يعتبر فحص الشبكة (Network Scanning) ركيزة أساسية في مرحلة جمع المعلومات لمهندسي الأمن السيبراني ومديري الأنظمة. تتيح هذه العملية كشف الأجهزة النشطة داخل النطاق، ومعرفة المنافذ المفتوحة (Open Ports)، وتحديد نوعية الخدمات التي تعمل عليها، مما يسهل رصد نقاط الضعف وإغلاقها قبل استغلالها من قبل المهاجمين.

1. بيئة العمل وتحميل المكتبات المطلوبة

يعتمد السكريبت المرفق بشكل رئيسي على نظامين لإدارة البيانات وتدفق الحزم الإلكترونية:

- مكتبة Scapy:** مكتبة خارقة وقوية في لغة بايثون تتيح للمطورين إمكانية إرسال، واستقبال، وتعديل حزم الشبكة (Network Packets) بمستويات منخفضة تفوق قدرات المقابس التقليدية.
- مكتبة Socket المدمجة:** مكتبة أساسية في بايثون تستخدم لإنشاء اتصالات برمجية مباشرة عبر بروتوكولات مثل TCP و UDP لفحص توافر الخدمات.

خطوات التثبيت البرمجي:

لإعداد البيئة، يجب أولاً تثبيت مكتبة Scapy عبر مدير الحزم pip. افتح واجهة الأوامر (Terminal / CMD) ونفذ الأمر التالي:

```
pip install scapy
```

مكتبة Scapy تحتاج إلى برنامج وسيط لالتقاط الحزم من كارت الشبكة مباشرة. إذا واجهت خطأ أثناء التشغيل على نظام ويندوز، يجب تحميل وتثبيت أداة Npcap (أو WinPcap) وتفعيل خيار *"Install Npcap with WinPcap API-compatible Mode"* أثناء التنصيب.

2. كود السكريبت البرمجي المتكامل (Python Source Code)

فيما يلي الكود المصدري الكامل للسكريبت. يرجى حفظه في ملف باسم `net_scanner.py`:

```
import socket
import sys
from datetime import datetime
from scapy.all import ARP, Ether, srp

# قائمة بالمنافذ الشائعة والخدمات المرتبطة بها مع الثغرات المحتملة لغرض الفحص
COMMON_PORTS = {
    21: {"service": "FTP", "vulnerability": "Anonymous أنظمة قديمة قد تحتوي على ثغرة Login أو تسريب أوامر"},
    22: {"service": "SSH", "vulnerability": "(Brute Force) عرضة لهجمات التخمين المكثف في حال ضعف كلمة المرور."},
    23: {"service": "Telnet", "vulnerability": "بروتوكول غير مشفر بالكامل! يتم إرسال البيانات والاعتمادات بنص واضح."},
    80: {"service": "HTTP", "vulnerability": "قد يستضيف تطبيقات تحتوي على ثغرات برمجية (XSS, SQLi) أو إعدادات خاطئة."},
    443: {"service": "HTTPS", "vulnerability": "تحقق من استخدام شهادات تشفير منتهية أو بروتوكولات قديمة مثل SSLv3."},
    445: {"service": "SMB", "vulnerability": "خطير جداً! عرضة لثغرات تجاوز السعة مثل EternalBlue (MS17-010) في حال عدم التحديث"}
}

def scan_local_network(ip_range):
    """اكتشاف الأجهزة المتصلة بالشبكة المحلية باستخدام حزم ARP Requests
    print(f"[*] {ip_range}: جاري بدء فحص الشبكة المحلية للنطاق")
    try:
        # بناء حزمة طلب ARP لتحديد الأجهزة النشطة
        arp = ARP(pdst=ip_range)
        ether = Ether(dst="ff:ff:ff:ff:ff:ff") # بث عام لجميع الأجهزة
        packet = ether/arp

        # إرسال الحزم واستقبال الردود عبر الطبقة الثانية Layer 2
        result = srp(packet, timeout=3, verbose=0)[0]

        devices = []
```

```

        for sent, received in result:
            devices.append({'ip': received.psrc, 'mac': received.hwsrc})
        return devices
    except Exception as e:
        print(f"[-] حدث خطأ غير متوقع أثناء فحص الشبكة {e}")
        return []

def scan_ports(ip):
    """فحص المنافذ المفتوحة لجهاز مستهدف ومطابقتها بالثغرات المحتملة"""
    print(f"\n[+] بدء فحص المنافذ والخدمات للجهاز {ip}")
    open_ports_found = False

    for port, info in COMMON_PORTS.items():
        # إنشاء مأخذ اتصال TCP قياسي
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(1.0) # مهلة الاستجابة ثانية واحدة لكل منفذ

        # محاولة الاتصال بالمنفذ (الرمز 0 يعنى نجاح الاتصال والميناء مفتوح)
        result = s.connect_ex((ip, port))
        if result == 0:
            open_ports_found = True
            print(f"    [{!}] المنفذ المفتوح {port} -> الخدمة: ({info['service']})")
            print(f"    ⚠️ التهديد المحتمل: {info['vulnerability']}")
            s.close()

    if not open_ports_found:
        print(f"    [-] لم يتم العثور على أي منافذ شائعة مفتوحة على هذا الجهاز")

def main():
    print("="*65)
    print(f"          أداة فحص الشبكات واستكشاف الثغرات الأمنية          ")
    print("="*65)

    # طلب إدخال النطاق المستهدف من المستخدم
    target_network = input("أدخل نطاق الشبكة المراد فحصها (مثال: 192.168.1.1/24): ")

    start_time = datetime.now()

    # المرحلة الأولى: كشف المضيفين (Host Discovery)
    devices_list = scan_local_network(target_network)

    print(f"\n[+] جهاز نشط على الشبكة ({len(devices_list)}) تم اكتشاف")
    for device in devices_list:
        print(f"    IP: {device['ip']} \t عنوان MAC: {device['mac']}")

    print("-"*65)

```

```
# المرحلة الثانية: فحص المنافذ والثغرات
if devices_list:
    run_port_scan = input("\nهل ترغب في فحص المنافذ والثغرات للأجهزة المكتشفة (y/n): ")
    if run_port_scan.lower() == 'y':
        for device in devices_list:
            scan_ports(device['ip'])
    else:
        print("[-] لم يتم العثور على أجهزة نشطة للاستمرار في الفحص")

end_time = datetime.now()
print(f"\n[*] الوقت المستغرق: {end_time - start_time}")

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n[-] تم إلغاء الفحص فوراً بواسطة المستخدم")
        sys.exit()
```

3. الشرح الهندسي الدقيق لآلية عمل الكود

أولاً: مرحلة اكتشاف الأجهزة (Host Discovery) via ARP

عند فحص شبكة محلية، الطريقة التقليدية (مثل إرسال حزم Ping / ICMP Echo) غالباً ما تفشل بسبب قيام الجدران النارية الحديثة بحظرها. لذلك، يعتمد هذا السكريبت على بروتوكول ARP (Address Resolution Protocol) في الطبقة الثانية لنموذج OSI. يقوم السكريبت بالآتي:

1. يُنشئ حزمة طلب (ARP Request) يسأل فيها: "من يمتلك عنوان IP هذا؟".
2. يضع الحزمة داخل إطار إيثرنت (Ethernet Frame) موجه إلى العنوان الفيزيائي العام `ff:ff:ff:ff:ff:ff`. مما يجبر جميع أجهزة الشبكة على استقبال الحزمة (Broadcast).
3. تستجيب الأجهزة النشطة فقط عبر إرسال عنوان MAC الخاص بها. يتم تجميع هذه البيانات عبر الدالة `srp()` ومعالجتها لإعادتها كقائمة منظمة تحتوي على عناوين IP و MAC الحقيقية.

ثانياً: مرحلة فحص المنافذ (Port Scanning) via TCP Connect

بعد حصر الأجهزة النشطة، ينتقل السكريبت إلى فحص المنافذ باستخدام `TCP Connect Scan` عبر دالة `socket.connect_ex()`:

- يقوم السكريبت بمحاولة إنشاء مصافحة ثلاثية (Three-Way Handshake) كاملة مع المنفذ المحدد.
- إذا كان المنفذ **مفتوحاً (Open)**، تستجيب الأداة البرمجية للنظام بالقيمة الرقمية `0` ويتم إغلاق الاتصال فوراً لتجنب إحداث ضغط.
- إذا أرجعت الدالة أي قيمة أخرى (مثل أخطاء المهلة أو الرفض)، فهذا يعني أن المنفذ مغلق أو محمي بجدار ناري.

ثالثاً: آلية تقييم الثغرات (Vulnerability Assessment)

يعتمد السكريت على قاعدة بيانات محلية مبسطة (قاموس COMMON_PORTS). بمجرد اكتشاف منفذ مفتوح، يقوم السكريت بربط هذا المنفذ بالثغرات الأمنية التاريخية والشائعة المرتبطة به لإرشاد الفاحص إلى مواطن الخطر البديهية بالمنظومة.

4. طريقة التشغيل وإدارة الصلاحيات عبر الأنظمة المختلفة

يتطلب الفحص المباشر للشبكات وصياغة الحزم المخصصة (Raw Packets) صلاحيات مرتفعة على كروت الشبكة. فيما يلي الطريقة الصحيحة لتشغيل السكريت عبر الأنظمة:

اسم نظام التشغيل	الأمر البرمجي لتشغيل كمنسؤول	متطلبات خاصة إضافية بالبيئة
Linux (Ubuntu/Kali)	<code>sudo python3 net_scanner.py</code>	لا توجد متطلبات، يعمل مباشرة من الطرفية.
macOS	<code>sudo python3 net_scanner.py</code>	يتطلب إعطاء صلاحية للـ Terminal للوصول لكارت الشبكة.
Windows 10 / 11	تشغيل CMD ك Administrator ثم: <code>python net_scanner.py</code>	تثبيت مكتبة/برنامج Npcap ضروري جداً لالتقاط الحزم.

شرح خطوات التشغيل التفصيلية:

- افتح واجهة الأوامر الخاصة بنظامك بالصلاحيات الإدارية القصوى (Root/Administrator).
- انتقل إلى المجلد الذي قمت بحفظ الملف داخله باستخدام أمر الانتقال (مثال: `cd Desktop`).
- نفذ أمر التشغيل المناسب لنظامك كما هو موضح بالجدول أعلاه.
- سيطلب منك السكريت إدخال النطاق (Subnet range). إذا كان الآي بي الخاص بجهازك `192.168.1.15`، فإن النطاق الافتراضي لشبكتك غالباً سيكون `192.168.1.1/24` أو يمكنك كتابة نطاق مخصص للفحص.
- بعد انتهاء عملية كشف الأجهزة، اضغط على زر `y` للموافقة على بدء فحص الثغرات والمنافذ بدقة لكل هدف مكتشف على حدة.